

Phase 4 - All Incident Scenarios



Your group will assign ONE scenario. This page covers all three so you're prepared for any of them.

Scenario A: Security Group Block (RECOMMENDED)

Why this is easiest: Quick to execute, instant effect, easy to reverse, doesn't touch credentials or DB state.

Step A.1 — Find Your Security Group IDs

```
# Find RDS Security Group
aws rds describe-db-instances \
  --db-instance-identifier lab-mysql \
  --query "DBInstances[0].VpcSecurityGroups[*].VpcSecurityGroupId" \
  --output text

# Find EC2 Security Group (note the source-group used in RDS SG rules)
aws ec2 describe-security-groups \
  --group-ids <RDS_SG_ID> \
  --query "SecurityGroups[0].IpPermissions"
```

Step A.2 — Block the Connection (Revoke Rule)

```
aws ec2 revoke-security-group-ingress \
  --group-id <RDS_SG_ID> \
```

```
--protocol tcp \  
--port 3306 \  
--source-group <EC2_SG_ID>
```



SAVE THIS COMMAND FOR RECOVERY:

```
aws ec2 authorize-security-group-ingress \  
  --group-id <RDS_SG_ID> \  
  --protocol tcp \  
  --port 3306 \  
  --source-group <EC2_SG_ID>
```

Step A.3 — Trigger the Error

```
curl http://<EC2_PUBLIC_IP>/list
```

Expected: Connection timeout or error message.

What You'll See in Logs

```
ERROR: Database connection failed - (2003, "Can't connect to  
MySQL server on '...' (timed out)")
```

Recovery Command (Phase 5)

```
aws ec2 authorize-security-group-ingress \  
  --group-id <RDS_SG_ID> \  
  --protocol tcp \  
  --port 3306 \  
  --source-group <EC2_SG_ID>
```

Scenario B: Stop RDS Instance

Why this is harder: Takes 5-10 minutes to stop, 5-10 minutes to start. Total incident window ~15-20 minutes.

Step B.1 — Stop the RDS Instance

```
aws rds stop-db-instance --db-instance-identifier lab-mysql
```

Step B.2 — Monitor Status

```
aws rds describe-db-instances \
  --db-instance-identifier lab-mysql \
  --query "DBInstances[0].DBInstanceStatus"
```

Wait for status: `stopped`

Step B.3 — Trigger the Error

```
curl http://<EC2_PUBLIC_IP>/list
```

Expected: Connection refused or timeout.

What You'll See in Logs

```
ERROR: Database connection failed - (2003, "Can't connect to
MySQL server on '...' (Connection refused)")
```

Recovery Command (Phase 5)

```
aws rds start-db-instance --db-instance-identifier lab-mysql

# Wait for it to become available
```

```
aws rds wait db-instance-available --db-instance-identifier lab-mysql
```



Timing: RDS start can take 5-10 minutes. Plan accordingly.

Scenario C: Credential Mismatch (Change Password)

Why this is tricky: Tests your understanding of credential flow. App has old password cached, DB expects new password.

Step C.1 — Break the Credentials

```
aws secretsmanager update-secret \  
  --secret-id lab/rds/mysql \  
  --secret-string '{"username":"admin","password":"WRONGPASSWORD","host":"<RDS_ENDPOINT>","port":"3306"}'
```



Important: This puts a WRONG password in Secrets Manager. The DB still expects the CORRECT password.

Step C.2 — Restart the App (To Pick Up New Secret)

```
# SSH into EC2  
sudo systemctl restart rdsapp
```

Step C.3 — Trigger the Error

```
curl http://<EC2_PUBLIC_IP>/list
```

Expected: Authentication failure.

What You'll See in Logs

```
ERROR: Database connection failed - (1045, "Access denied for user 'admin'@'...' (using password: YES)")
```

Recovery Command (Phase 5)

```
aws secretsmanager update-secret \  
  --secret-id lab/rds/mysql \  
  --secret-string '{"username":"admin","password":"<CORRECT_PASSWORD>","host":"<RDS_ENDPOINT>","port":"3306"}'
```

```
# SSH into EC2 and restart app  
sudo systemctl restart rdsapp
```



Key Lesson: This scenario teaches why apps caching credentials is dangerous during rotation. The app held the wrong password because it fetched it once at startup.

After Any Scenario: Verify Incident Detection

Check ERROR Logs Appeared

```
aws logs filter-log-events \  
  --log-group-name /aws/ec2/lab-rds-app \  
  --filter-pattern ERROR
```

```
--filter-pattern "ERROR" \  
--limit 10
```

Check Alarm Fired

```
aws cloudwatch describe-alarms \  
--alarm-names lab-db-connection-failure-alarm \  
--query "MetricAlarms[0].StateValue"
```



Expected: "ALARM"

Check Alarm History

```
aws cloudwatch describe-alarm-history \  
--alarm-name lab-db-connection-failure-alarm \  
--history-item-type StateUpdate \  
--max-records 5
```

Expected: Transition from OK → ALARM

Scenario Comparison Table

Aspect	Scenario A (SG)	Scenario B (RDS Stop)	Scenario C (Password)
Time to Break	Instant	5-10 min	30 sec + restart
Time to Recover	Instant	5-10 min	30 sec + restart
Error Message	"timed out"	"Connection refused"	"Access denied"
Difficulty	Easy	Easy but slow	Medium
Real-World Analog	Firewall change	DB maintenance	Secret rotation failure